

Introduction to Programming COMP101 2025-26

Lab-04 Supervised

Objectives:

Exercise-01 Pseudocode Selection

OR

Work Assignment-02

Exercise-01: Selection

Produce a Python program to present a menu to the user.

Think about the design to make it easy for a user and to keep them informed of what is going on, i.e.

What will happen here (eventually after more development) is the user can select an option and practice maths under each of the headings. However, for now, they will be presented with a ‘stub’ – see below and the program will and after each option.

Main Menu Maths

A. Addition
D. Division
M. Multiplication
S. Subtraction
X Exit Program

Enter an option (A, D, M, S or X:)

a) No validation is required (yet) so assume user inputs correct data as indicated in the menu above.

b) User can input ‘A’ or ‘a’ etc. at the input prompt.

How do you propose to deal with this?

HINT: if statement could use compound Boolean tests

c) You can simplify your if statement in (b) above by converting user input to upper or lower case (your choice) using an appropriate function?

Operation:

1 If option A or a, D or d, M or m, S or s is input use a ‘stub’ to tell the user:

i) the option they selected

ii) a ‘stub message’ telling them the option is under development

The program can finish – it does not need to iterate back to the menu (yet).

2 If option X or x is entered:

Output a confirmation message that the program is ending and exit the program.

Exercise-01: Selection **SOLUTION**

#Lab-04 Solution.py

```
print ('Main Menu Maths')
print ('-----')
print ('A. Addition')
print ('D Division')
print ('M Multiplication')
print ('S Subtraction')
print ('X Exit Program')
print ()
option = input('Enter an option (A, D, M, S or X): ')

if (option == 'A' or option == 'a'):
    print ('Option-A selected')
    print ('Maths under development')
    input('Press return to exit')

elif (option == 'D' or option == 'd'):
    print ('Option-D selected')
    print ('Division under development')
    input('Press return to exit')

elif (option == 'M' or option == 'm'):
    print ('Option M selected')
    print ('Multiplication under development')
    input('Press return to exit')

elif (option == 'S' or option == 's'):
    print ('Option S selected')
    print ('Subtraction under development')
    input('Press return to exit')

else:
    print('Option X selected')
    input ('Press return to end program')

print ('Program has ended')
```

Notes: Exercise-01 solution:

a) Think about the user. It seems more ‘natural’ to have menu A to S alphabetically rather than A, B, C, D or 1,2,3,4

b) Each option executes once only (for now) and the program ends. Eventually we will iterate the options so that when an option completes, we return to the main menu – but not just yet.

c) Note the ‘one-if’ ‘one-else’ and ‘as many elif’ structure. The single else ‘captures the exception’ – it executes when all other options have been exhausted, hence it does not specifically need to test as:
else option == ‘X’ or option == ‘x’

d) The messages will need expanding eventually. For option A, D, M and S they will eventually return user to the main menu (at the moment execution stops), so they will eventually say ‘Press return to go back to main menu’.

For option X, we tell the user this will end the program – the menu will not re-appear after this option so we tell the user ‘Press return to exit the program’

Basically, a ‘good’ menu should have one entry point and one exit point, i.e. the program can only be accessed via the main menu (options A, D, M and S) and can only end via the main menu (option X).

To make this happen we will need the third programming construct – ITERATION – in the next lecture sets.

e) You could amend the program and remove the compound expressions and replace them using a function to alter the input case (uppercase to lowercase or vice-versa).

e.g.

```
option = input('Enter an option (A, D, M, S or X): ')
option = (option.upper())
if option == 'A': #only need test for uppercase 'A' as its been converted
    print ('Option-A selected')
etc
```